

生产级 FIX 核心问题与回答

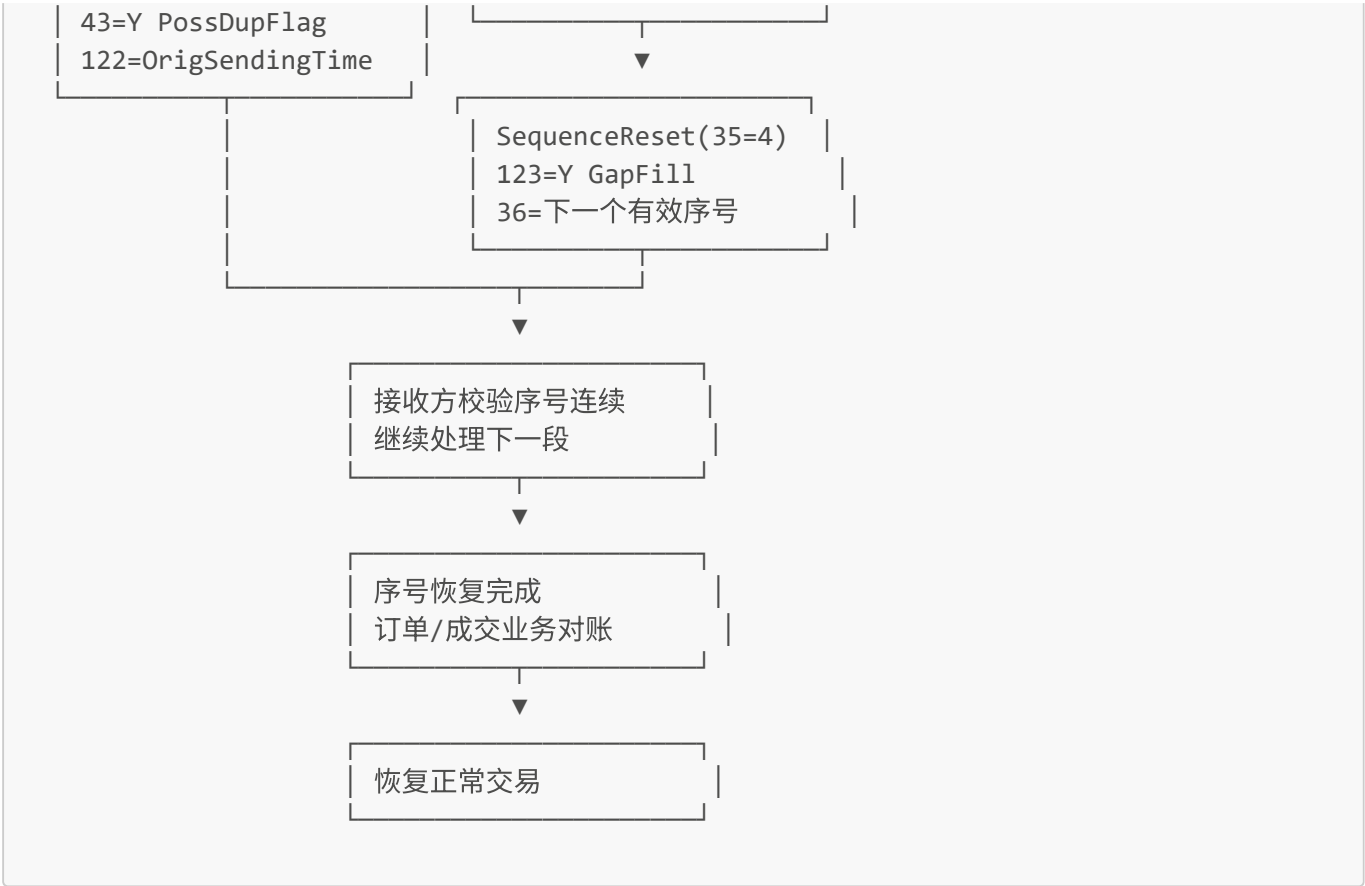
本文整理生产环境中常见的 FIX 订单与会话问题。每个问题只保留两部分：架构图和精简回答。

机构交易架构设计

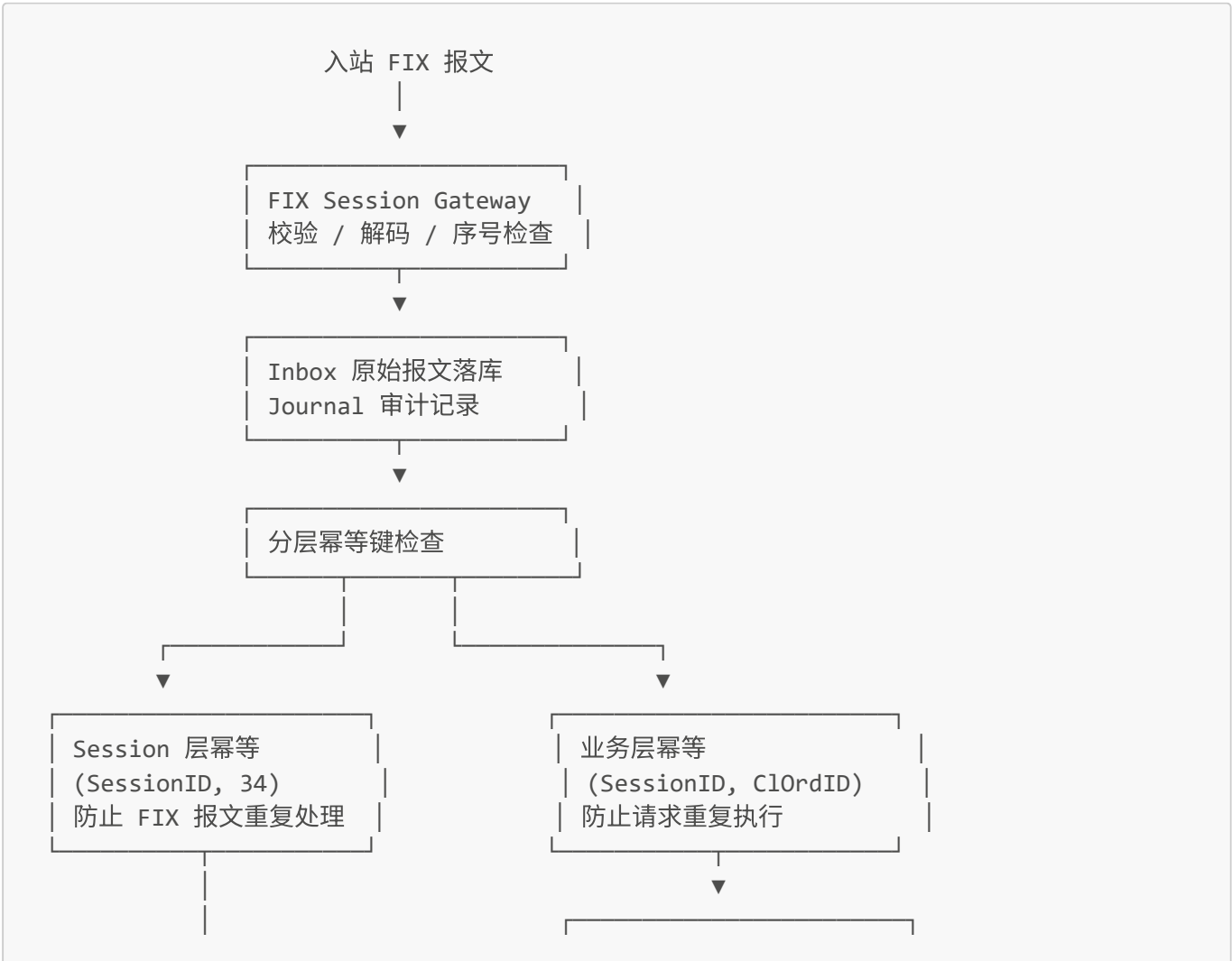
https://technologynova.org/%e4%bb%8etag-35d%e5%88%b0tag-358%ef%bc%9a%e6%b7%b1%e5%ba%a6%e8%a7%a3%e5%89%96fix%e5%8d%8f%e8%ae%ae%e5%9c%a8%e8%ae%a2%e5%8d%95%e7%ae%a1%e7%90%86%e7%b3%bb%e7%bb%9f%ef%bc%88oms%ef%bc%89%e4%b8%ad/

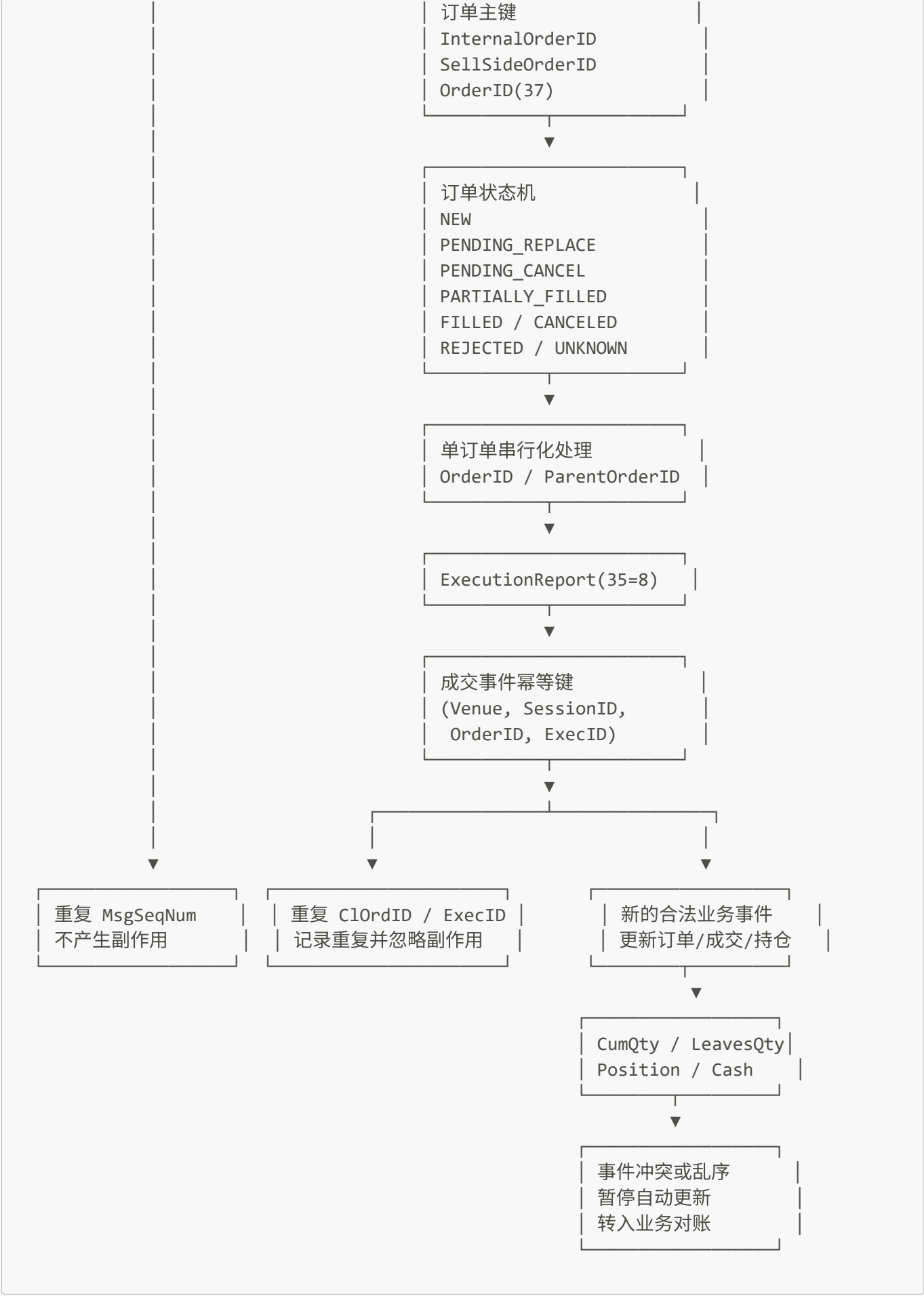
一：消息重传框架



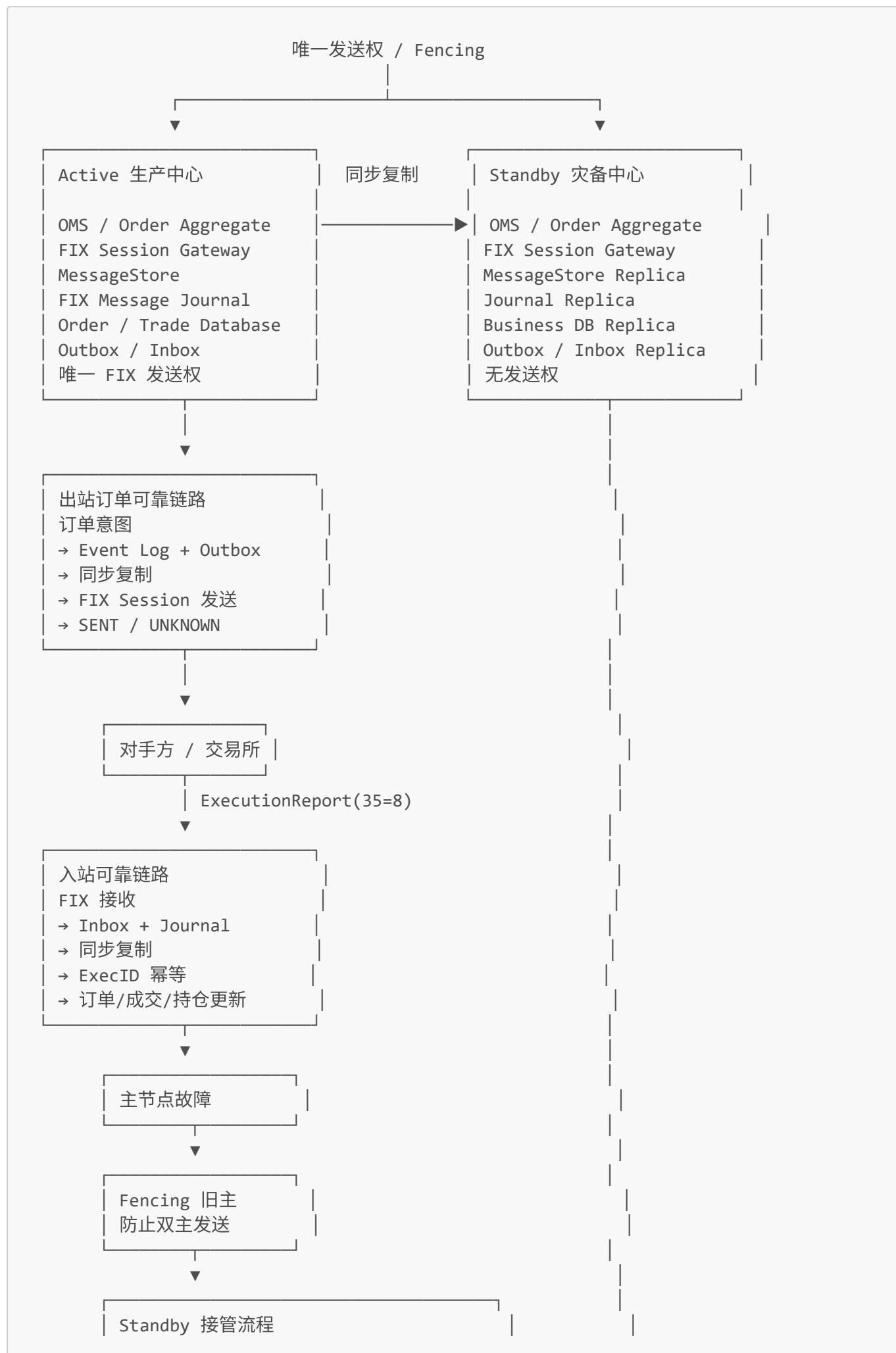


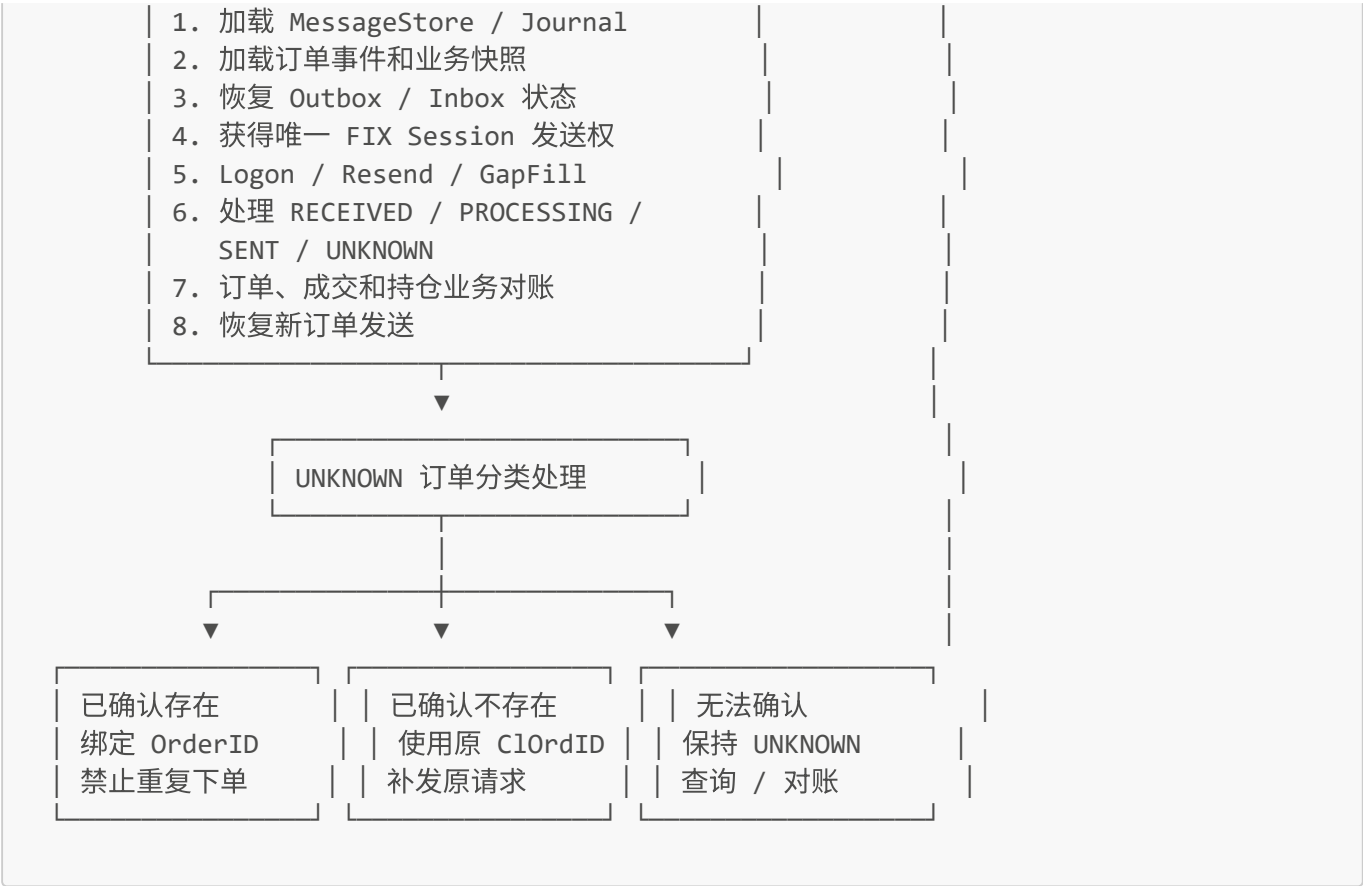
二：业务幂等框架





三：灾备处理框架



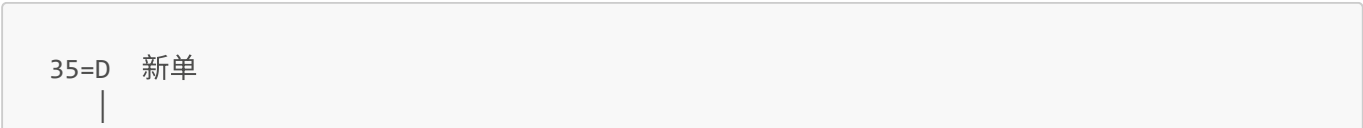


Top10 核心 MsgType

下面是买方、卖方和交易所订单链路中最常用、最值得优先掌握的 10 个 MsgType(35):

排名	MsgType	消息	主要作用
1	A	Logon	建立或恢复 FIX Session
2	0	Heartbeat	保持 Session 活跃
3	1	TestRequest	请求对方立即返回 Heartbeat
4	2	ResendRequest	请求重传缺失序号范围
5	4	SequenceReset	序号重置或 GapFill
6	5	Logout	正常关闭 Session
7	D	NewOrderSingle	发送新订单
8	8	ExecutionReport	返回订单状态、成交、撤单等结果
9	F	OrderCancelRequest	请求撤销订单
10	G	OrderCancelReplaceRequest	请求改单

核心订单链路：



- ├ 35=8 NEW / PARTIAL / FILLED
- ├ 35=G 改单请求
- └ 35=F 撤单请求
 - ├ 35=8 撤单状态回报
 - └ 35=9 撤单被拒

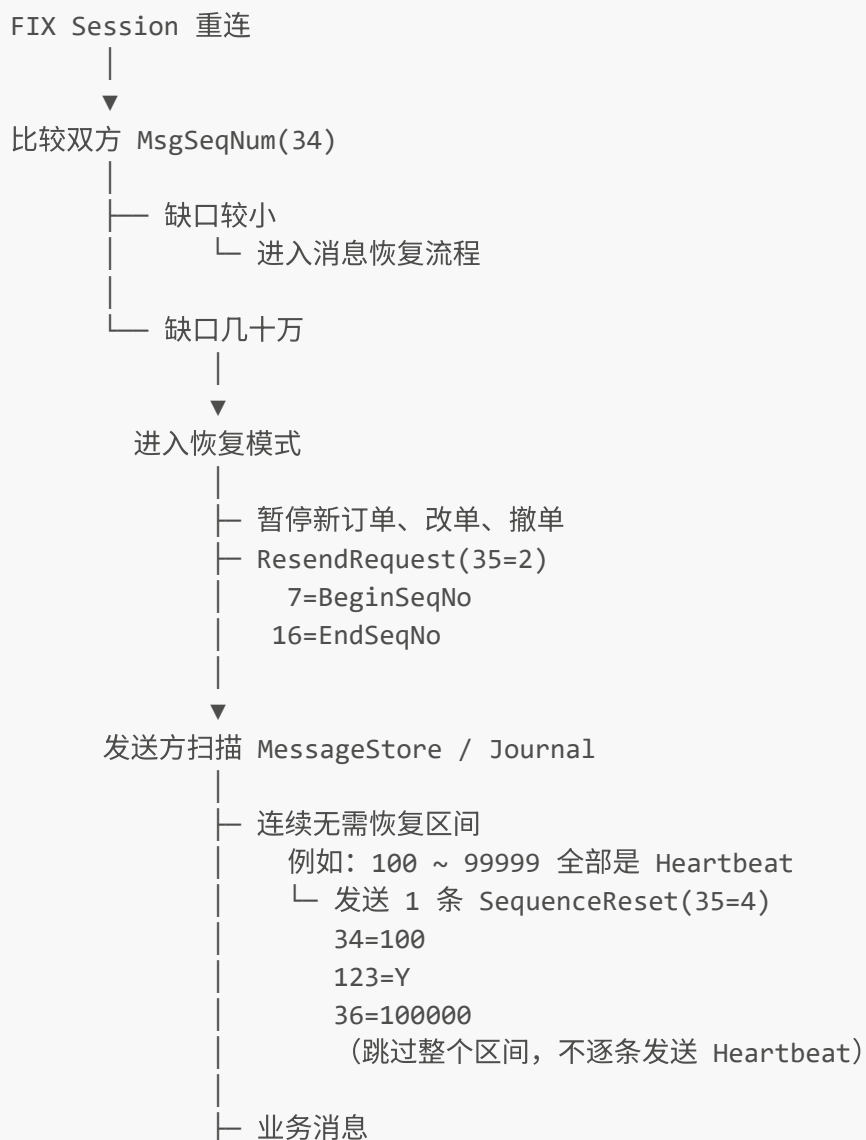
补充两个很重要的消息：

35=3 Reject
= Session 层消息被拒绝

35=9 OrderCancelReject
= 撤单请求被拒绝

1. 断线重连后 MsgSeqNum 相差几十万，应该如何处理？

架构图





精简回答

1. 按范围请求恢复：缺口出现后暂停新订单、改单和撤单，使用 `ResendRequest(35=2)` 请求序号区间：

```
7=BeginSeqNo
16=EndSeqNo
```

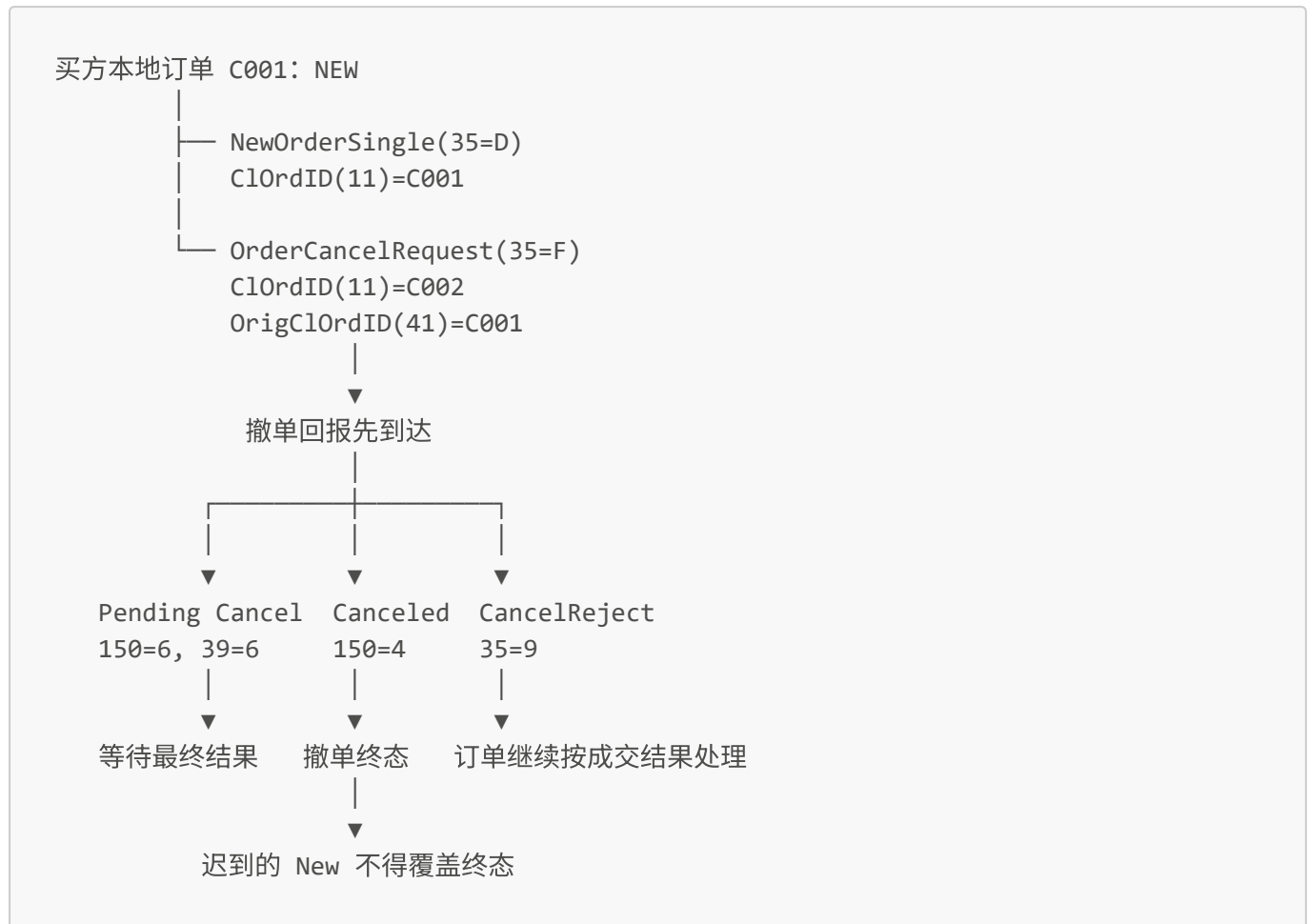
2. 业务消息必须恢复：35=D、35=8、35=F、35=G 等关键消息原样重传，保留原 `MsgSeqNum(34)`，并设置 `PossDupFlag(43)=Y`、`OrigSendingTime(122)`；也可以通过业务对账确认后恢复。
3. 非业务消息使用 GapFill：连续的 `Heartbeat`、`TestRequest` 和可恢复行情增量不逐条重传，而是用一条 `SequenceReset(35=4, 123=Y, 36=NewSeqNo)` 跳过整个连续区间。35=4 不是逐条替代 `Heartbeat`，而是一条区间跳转控制消息。
4. 混合区间按业务消息分段：扫描 `MessageStore / Journal`，遇到连续非业务区间先发送一条 `GapFill`，遇到业务消息再原样重传。例如：

```
35=4  跳过 100~99999
35=D  重传 100000
35=4  跳过 100001~199999
35=8  重传 200000
```

5. 恢复后完成校验：序号对齐不代表业务状态已恢复，必须完成订单、成交和撤单结果对账后，才能恢复正常交易；历史 `Heartbeat` 可跳过，但实时 `Heartbeat` 仍需正常发送。

2. 一笔订单和对应撤单同时发出，先收到撤单 ExecutionReport，应该如何处理？

架构图

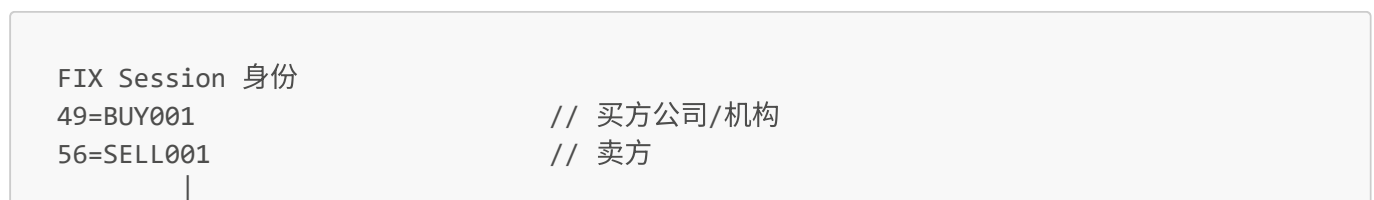


精简回答

- 使用 `ClOrdID(11)=C002` 标识当前撤单请求，使用 `OrigClOrdID(41)=C001` 指向原订单，并通过 `OrderID(37)` 关联卖方订单。
- 收到 `ExecType=Pending Cancel(6)`：进入待撤状态；收到 `ExecType=Canceled(4)`：撤单成功并进入终态。
- 收到 `OrderCancelReject(35=9)`：撤单失败，不能当作撤单成功；如果交易所先成交，撤单通常会被拒绝。
- 订单进入 `Canceled` 或 `Filled` 等终态后，迟到的 `New` 回报不能覆盖终态。
- 最终状态必须由订单状态机和交易所实际处理顺序决定，不能简单按回报到达顺序覆盖。

3. 如何把一个公司的订单与资金账号关联起来？

架构图





精简回答

1. **SenderCompID(49)** 表示 FIX Session 层的公司或机构身份，**Account(1)** 表示本笔订单使用的资金或交易账号。
2. 账户主数据通过 **Account(1)** 关联公司、基金、策略、风控和结算信息，例如：

```

49=BUY001
11=C001
1=FUND-A-TRADING
  
```

3. 卖方通过 **BUY001 + FUND-A-TRADING** 查找具体公司和账户；一个公司管理多个基金或子账户时，每笔订单都应明确填写 **Account(1)**。
4. 需要表达基金、执行方、清算方等更细粒度参与者时，使用 **Parties** 重复组：

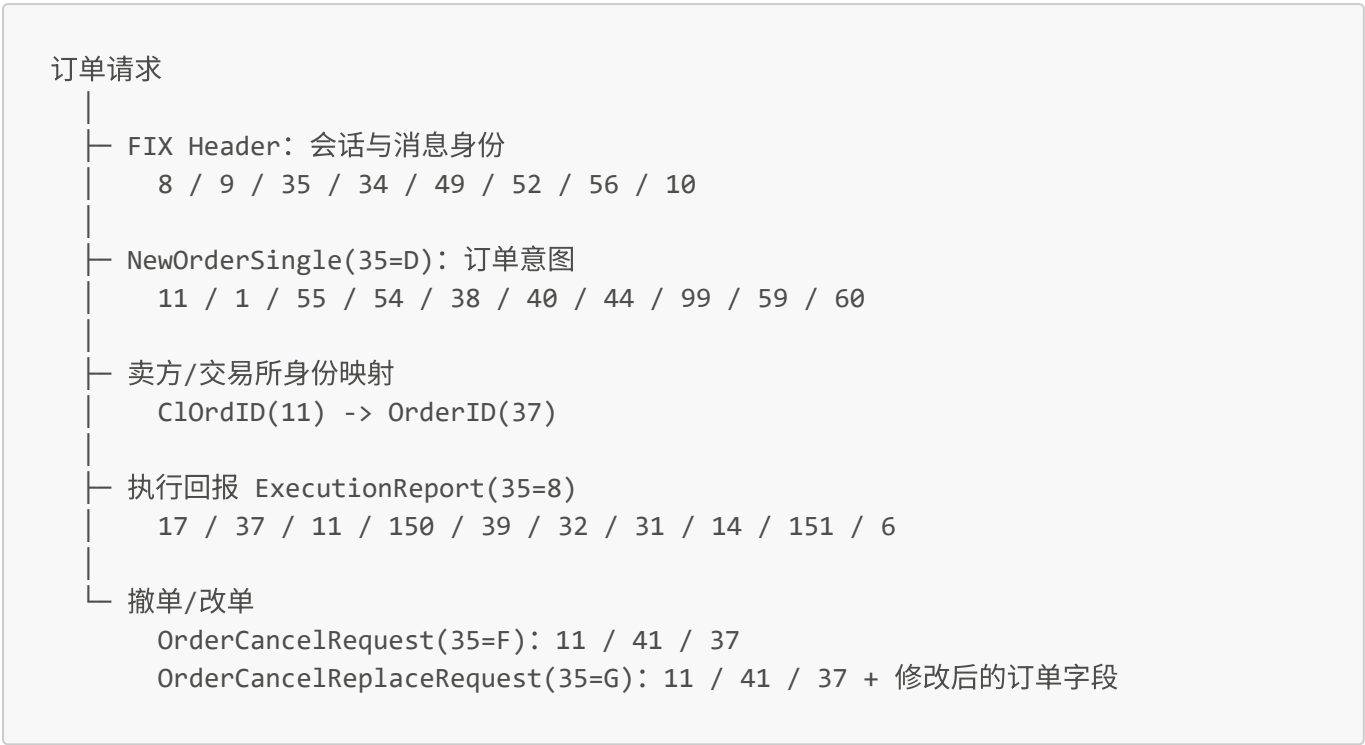
```

453=NoPartyIDs
448=PartyID
447=PartyIDSource
452=PartyRole
  
```

5. FIX 不会自动判断账号属于哪个公司，双方必须通过 Session 身份、账号约定和账户主数据完成关联。

4. 一笔下单涉及哪些主要要素，以及对应的 Tag/Value?

架构图



精简回答

1. 说明：以下是常用的 FIX 4.2 订单要素；是否必填以 `FIX42.xml`、对手方协议和订单类型为准。
2. 会话 Header：

Tag	字段	示例 Value	含义
8	BeginString	FIX.4.2	FIX 版本
9	BodyLength	...	Body 长度，由引擎计算
35	MsgType	D	NewOrderSingle
34	MsgSeqNum	101	当前方向消息序号
49	SenderCompID	BUY001	发送方机构身份
52	SendingTime	20260908-09:30:00.000	发送时间
56	TargetCompID	SELL001	接收方机构身份
10	Checksum	...	报文校验和，由引擎计算

3. NewOrderSingle(35=D)：

Tag	字段	示例 Value	含义
11	ClOrdID	C001	当前发单方客户订单号
1	Account	FUND-A-TRADING	资金/交易账号
21	HandlInst	1	处理指示

Tag	字段	示例 Value	含义
55	Symbol	600000	证券/合约
54	Side	1	1=Buy、2=Sell
38	OrderQty	100	订单数量
40	OrdType	2	1=Market、2=Limit
44	Price	10.50	限价，限价单通常需要
99	StopPx	10.00	止损价，止损单使用
59	TimeInForce	0	0=Day、1=GTC、3=IOC
60	TransactTime	20260908-09:30:00.000	交易时间
114	LocateReqd	N	卖空定位需求，按协议使用
453	NoPartyIDs	...	参与方重复组数量
448	PartyID	FUND-A	参与方标识，位于 Parties 组内
447	PartyIDSource	D	参与方标识来源
452	PartyRole	...	参与方角色

4. ExecutionReport(35=8):

Tag	字段	示例 Value	含义
37	OrderID	S1001	接收方分配的订单号
17	ExecID	E001	本次执行事件编号
11	ClOrdID	C001	关联买方订单
41	OrigClOrdID	C001	关联原订单版本，改单/撤单场景使用
20	ExecTransType	0	FIX 4.2 执行交易类型
150	ExecType	0/1/2/4/6	New/Partial/Fill/Canceled/Pending Cancel
39	OrdStatus	0/1/2/4/6/8	New/Partial/Filled/Canceled/Pending Cancel/Rejected
32	LastShares	100	FIX 4.2 本次成交数量
31	LastPx	10.50	本次成交价格
14	CumQty	100	累计成交数量
151	LeavesQty	0	剩余数量
6	AvgPx	10.50	平均成交价格
1	Account	FUND-A-TRADING	关联资金/交易账号
58	Text	...	补充说明

5. 撤单与改单：

MsgType	消息	关键 Tag
F	OrderCancelRequest	11=ClOrdID、41=OrigClOrdID、37=OrderID、55=Symbol、54=Side
G	OrderCancelReplaceRequest	11、41、37，以及新的 38、40、44、59
9	OrderCancelReject	11、41、37、39、434、102

6. ExecType(150) 和 OrdStatus(39) 的区别：

字段	含义	示例
ExecType(150)	本次 ExecutionReport 表达的事件，即“这次发生了什么”	New、Partial Fill、Fill、Pending Cancel、Canceled
OrdStatus(39)	处理本次事件后订单的当前状态，即“订单现在是什么状态”	New、Partially Filled、Filled、Pending Cancel、Canceled

例如：

```
150=2  Fill
39=2   Filled
```

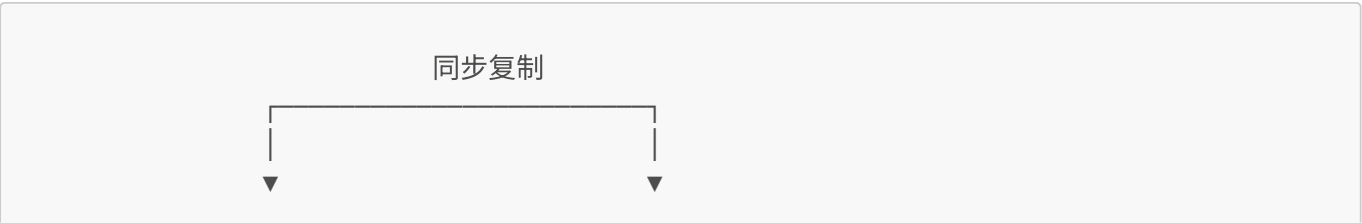
表示本次事件是完全成交，处理后订单当前状态为已成交。两者有时取值相同，但职责不同；ExecType 是事件，OrdStatus 是状态。

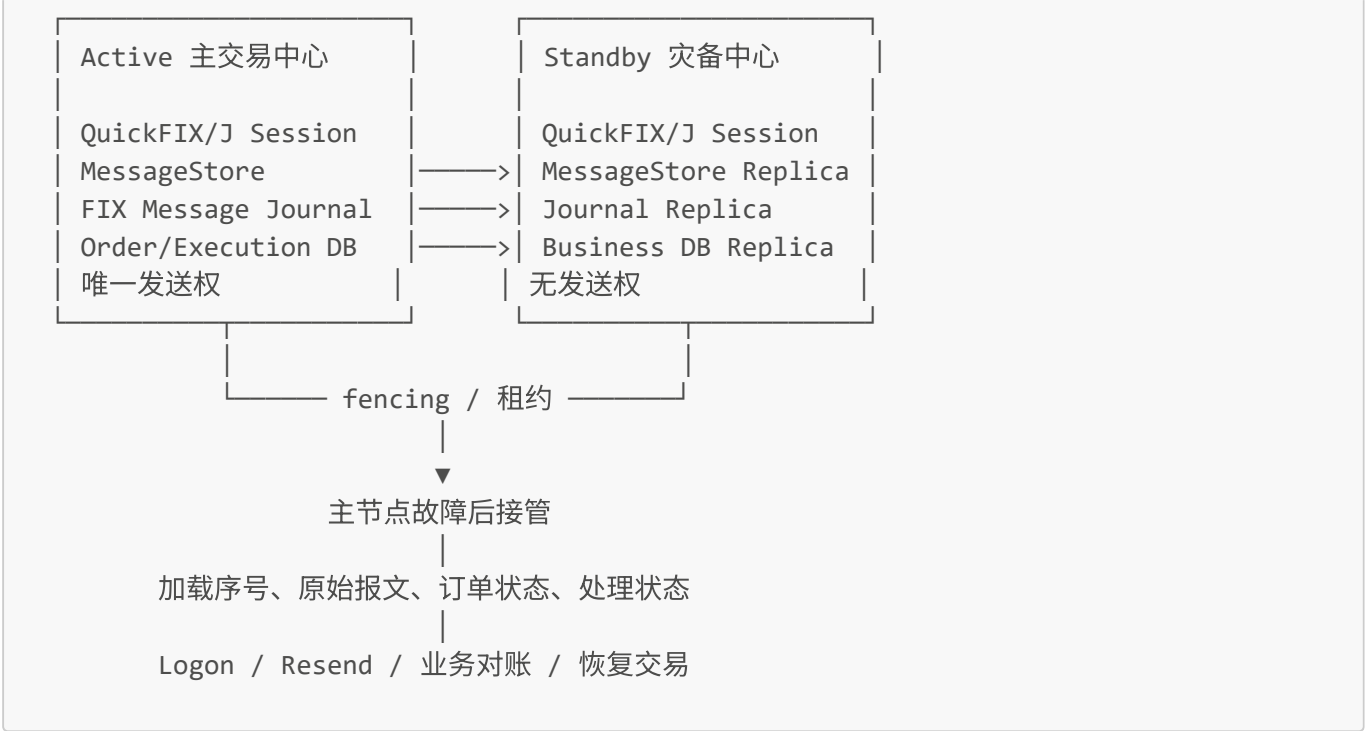
7. 核心关联：

Account(1)	= 资金账号
ClOrdID(11)	= 当前请求标识
OrigClOrdID(41)	= 原订单版本
OrderID(37)	= 接收方订单标识
ExecID(17)	= 执行事件标识
OrdStatus(39)	= 当前订单状态
ExecType(150)	= 本次状态事件

5. 灾备切换时，如何保存每一条消息并避免丢失或重复处理？

架构图





精简回答

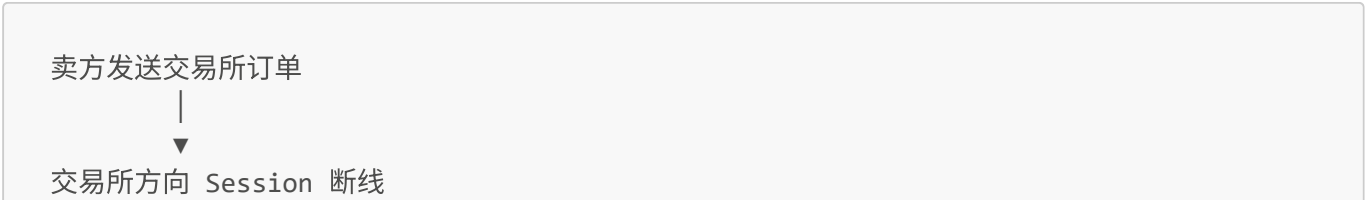
- 1. 分开保存三类数据：MessageStore 保存 FIX 历史报文用于 Session 重传；FIX Message Journal 保存每条入站、出站原始报文用于审计和重放；业务数据库保存订单、成交、持仓、资金和状态。
- 2. 进站消息先写入 Journal / Inbox 并同步复制到灾备，再完成业务处理、更新订单与成交，最后标记 PROCESSED；出站消息先持久化订单意图、原始报文和 ClOrdID，同步复制后发送，最后标记 SENT。
- 3. 主备切换时 fencing 旧主节点，由灾备获得唯一 Session 发送权，恢复 MessageStore 和业务数据库，处理 RECEIVED、PROCESSING、UNKNOWN 消息，并执行 FIX Logon、Resend 和业务对账。
- 4. 使用唯一键和幂等机制避免重复：

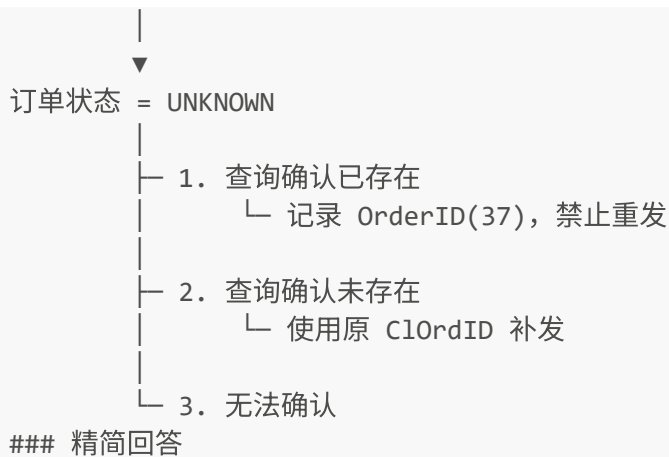
(SessionID, MsgSeqNum)
(SessionID, ExecID)
(OrderID, EventID)

- 5. RPO=0 时，消息必须同步持久化到灾备副本后才视为可靠；网络发送后宕机形成 UNKNOWN 时，必须查询交易所或对手方，不能直接生成新的 ClOrdID 重发。

6. 卖方已向买方返回 NEW，但到交易所的 Session 断线，如何避免重复下单？

架构图

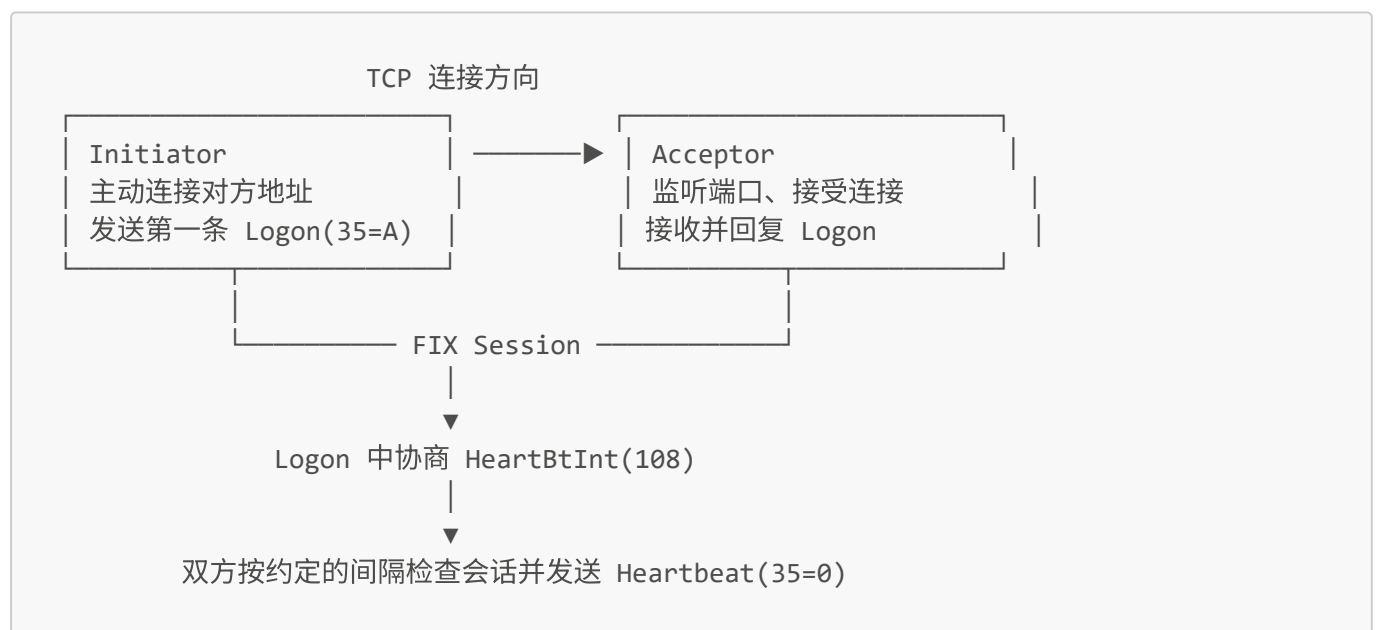




1. 买方收到卖方 `OrdStatus=NEW`，只说明卖方已接单，不代表交易所已接收；断线后卖方应将交易所方向订单标记为 `UNKNOWN`，保留原 `C1OrdID`。
2. 交易所确认订单已存在：记录交易所 `OrderID(37)`，绑定原订单，禁止重复下单。
3. 交易所明确确认不存在：使用原 `C1OrdID` 补发，不生成新的 `C1OrdID`。
4. 无法确认时：保持 `UNKNOWN`，暂停重发，继续查询或进行业务对账。
5. 核心原则：使用 `C1OrdID + 状态查询 + 幂等控制`，不能把本地发送成功当成交易所已接收。

7. Acceptor、Initiator 和 HeartBtInt(108) 如何区分？

架构图



精简回答

1. **Initiator** 主动连接对方地址，并发送第一条 **Logon(35=A)**；**Acceptor** 监听本地端口，接受连接并处理对方的 Logon。
2. 两者只表示 TCP 连接方向，不代表买方或卖方角色：买方和卖方都可以是 Initiator 或 Acceptor。
3. **HeartBtInt(108)** 是 FIX Session 的会话参数，不是业务字段；双方应在 Session 协议和配置中预先约定。

4. Initiator 通常在第一条 Logon 中提出 HeartBtInt(108); Acceptor 根据双方配置和协议校验、接受或拒绝不匹配的值。

5. 最终记忆:

```
Initiator  = 主动建立 TCP 连接
Acceptor   = 监听并接受 TCP 连接
HeartBtInt = 双方约定, Initiator 在 Logon 中提出, Acceptor 校验确认
```